Vulnerability Assessment Report

Website Security Audit — Altoro Mutual (`demo.testfire.net`)

Field	Details
Prepared by	Patrick Leon
Role	Cyber Security Intern — Future Interns Program

Field	Details
Date	4 March 2026
Target	`https://demo.testfire.net`
Assessment Type	Read-Only Passive Vulnerability Assessment
Classification	Confidential

Table of Contents

1. Executive Summary

This report presents the findings of a passive web application vulnerability assessment conducted against **Altoro Mutual** (`demo.testfire.net`), a publicly authorized test banking platform maintained by HCL Technologies. The assessment was commissioned as part of an ethical security learning exercise.

The assessment identified **15 alerts** across four risk tiers. The findings reveal a web application running on highly outdated server infrastructure and lacking critical browser-enforced security headers. While no high-severity exploitable vulnerabilities were detected within the read-only scope, the current configuration leaves the application and its users significantly exposed to client-side attacks, information leakage, and session hijacking.

Key Message for the Business: This website is operational but not secure by modern standards. The issues identified are the types that attackers routinely exploit against banking applications. Fixing them is not expensive — it primarily requires configuration changes and a server upgrade.

2. Scope & Rules of Engagement

Item	Value
Target Website	`https://demo.testfire.net`
IP Address	`65.61.137.117`
Authorization	Public test application — legally authorized for security testing by HCL Technologies
Assessment Type	Read-Only / Passive (no exploitation, no brute force, no injection)
Out of Scope	Any active exploitation, denial-of-service tests, or credential attacks

> **Ethical Note:** This assessment strictly adhered to a read-only, non-destructive approach. No data was accessed, modified, or exfiltrated.

3. Tools Used

Tool	Purpose
Nmap v7.95	Port scanning, OS detection, service & version enumeration
Nikto v2.5.0	Web server vulnerability scanner — missing headers, insecure methods
WhatWeb	Web technology fingerprinting — frameworks, cookies, server strings
OWASP ZAP v2.16.1	Automated passive web application vulnerability scanner
Kali Linux	Operating system used to conduct all security assessments
Browser (Firefox)	Manual visual inspection and screenshot capture

4. Methodology

The assessment followed a structured, industry-aligned methodology:

```
[Reconnaissance] → [Port Scanning] → [Technology Fingerprinting] → [Automated Scanning] → [Analysis & Reporting]
```

Phase 1 — Reconnaissance

Initial open-source reconnaissance to understand the application's surface area and confirm authorization.

Phase 2 — Port & Service Enumeration (Nmap)

```
nmap -sV -sC -T4 -O -oA evidence/nmap_scan demo.testfire.net
```

Identified open ports and running services including exact software versions.

Results:

- Port `80` (HTTP) — Apache Tomcat/Coyote JSP engine 1.1
- Port `443` (HTTPS) — Apache Tomcat/Coyote JSP engine 1.1
- Port `8080` (HTTP) — Open proxy behavior detected

Phase 3 — Technology Fingerprinting (WhatWeb)

```
whatweb -v https://demo.testfire.net | tee evidence/whatweb_results.txt
```

Confirmed the Java/Apache Tomcat stack, session cookie names (`JSESSIONID`), and HTTP header details.

Phase 4 — Web Vulnerability Scan (Nikto)

```
nikto -h https://demo.testfire.net -o evidence/nikto_results.txt
```

Detected missing security headers and dangerous HTTP methods.

Phase 5 — Application-Level Passive Scan (OWASP ZAP)

OWASP ZAP was configured as a proxy and ran a passive automated analysis of the application by browsing through its pages. ZAP identified **15 categories of alerts** without sending any exploitative requests.

5. Findings Summary

#	Vulnerability	Severity	Confidence	Found By
1	Absence of Anti-CSRF Tokens	Medium	High	OWASP ZAP
2	Content Security Policy (CSP) Header Not Set	Medium	High	OWASP ZAP
3	Missing Anti-Clickjacking Header (`X-Frame-Options`)	Medium	Medium	Nikto + ZAP
4	Insecure HTTP Methods Enabled (`PUT`, `DELETE`)	Medium	Low	Nikto
5	Server Version Disclosed in HTTP Header	Low	High	Nmap + Nikto + ZAP
6	Outdated Server Component (Apache Tomcat Coyote 1.1)	Low	High	Nmap + WhatWeb
7	Strict-Transport-Security (HSTS) Header Not Set	Low	High	OWASP ZAP
8	X-Content-Type-Options Header Missing	Low	Medium	Nikto + ZAP
9	Cookie Without `SameSite` Attribute	Low	High	OWASP ZAP

#	Vulnerability	Severity	Confidence	Found By
10	Cross-Domain JavaScript Source File Inclusion	Low	Low	OWASP ZAP
11	Secure Pages Include Mixed Content	Low	Medium	OWASP ZAP
12	Information Disclosure — Suspicious Comments in Source	Info	Medium	OWASP ZAP
13	Session Management Response Identified	Info	Medium	OWASP ZAP
14	Re-examine Cache-Control Directives	Info	Medium	OWASP ZAP
15	Modern Web Application Detection	Info	Medium	OWASP ZAP

Total: 0 High · 4 Medium · 7 Low · 4 Informational

6. Detailed Findings

FINDING-01 — Absence of Anti-CSRF Tokens

Field	Details
Risk	Medium
Confidence	High
OWASP Reference	A01:2021 – Broken Access Control
Affected URL	`https://demo.testfire.net/login.jsp`, `/feedback.jsp`, `/subscribe.jsp`

What is the issue? HTML forms on the website (login, feedback, subscription) do not use Anti-CSRF (Cross-Site Request Forgery) tokens.

Why does it matter? (Business impact) An attacker could embed a hidden form on a malicious website. When a logged-in customer visits that page, their browser silently

submits a forged banking request (e.g., a fund transfer) — without the customer ever knowing. This is a critical risk for financial applications.

How to fix it: Implement per-session, server-validated CSRF tokens on every form that performs a state-changing action. Modern frameworks include this natively (e.g., Spring Security's `_csrf` token).

FINDING-02 — Content Security Policy (CSP) Header Not Set

Field	Details
Risk	Medium
Confidence	High
OWASP Reference	A05:2021 – Security Misconfiguration
Affected URL	All pages on `https://demo.testfire.net`

What is the issue? No `Content-Security-Policy` HTTP header is returned in any server response.

Why does it matter? (Business impact) CSP is the primary browser-level defence against Cross-Site Scripting (XSS) attacks. Without it, if an attacker finds even a minor XSS flaw, there is no secondary browser barrier to prevent the execution of a malicious script that could steal all logged-in customers' session cookies and account data.

How to fix it: Configure the web server to return a CSP header such as `Content-Security-Policy: default-src 'self'; script-src 'self'`. Start restrictive and gradually relax as needed.

FINDING-03 — Missing Anti-Clickjacking Header (`X-Frame-Options`)

Field	Details
Risk	Medium
Confidence	Medium

Field	Details
OWASP Reference	A05:2021 – Security Misconfiguration
Affected URL	All pages

What is the issue? The `X-Frame-Options` HTTP header is absent from all server responses, confirmed by both Nikto and OWA SP ZAP.

Why does it matter? (Business impact) Attackers can embed this banking site inside an invisible `<iframe>` on a fake page. A user who thinks they're clicking a "Play Video" button may actually be clicking the "Confirm Transfer" button on the hidden banking site. This attack is called **Clickjacking** and is particularly dangerous for financial applications performing transactions.

How to fix it: Add the `X-Frame-Options: SAMEORIGIN` or `X-Frame-Options: DENY` header to all server responses via the Apache Tomcat configuration file, or use the modern equivalent: `Content-Security-Policy: frame-ancestors 'self'`.

FINDING-04 — Dangerous HTTP Methods Enabled (`PUT` / `DELETE`)

Field	Details
Risk	Medium
Confidence	Low
Found by	Nikto
Evidence	`Allow: GET, HEAD, POST, PUT, DELETE, OPTIONS`

What is the issue? The `OPTIONS` request to the server returns a list of allowed HTTP methods including `PUT` and `DELETE`, which are typically used to write or remove files on the server.

Why does it matter? (Business impact) If server-side authorization is insufficient, an attacker could upload a malicious script (a "web shell") to the server via `PUT`, gaining full remote code execution access, or delete critical application files causing a service outage.

How to fix it: In the Apache Tomcat/web server configuration, explicitly restrict allowed HTTP methods to only `GET` and `POST`. Reject all others with a `405 Method Not Allowed` response.

FINDING-05 — Server Version Disclosed

Field	Details
Risk	Low
Confidence	High
Evidence	`Server: Apache-Coyote/1.1` (visible in all HTTP response headers)

What is the issue? The exact web server software and version is openly broadcast in every HTTP response header.

Why does it matter? (Business impact) `Apache-Coyote/1.1` is a component from Apache Tomcat's legacy architecture (15+ years old). Publishing this version banner allows any attacker to immediately search for known, public CVE exploits targeting this exact version — dramatically reducing the time needed to stage an attack. This is a critical information leakage finding for a system running such an old version.

How to fix it: Configure the server to suppress or genericize the `Server` header (e.g., `Server: Apache` only). Additionally, strongly prioritize an urgent upgrade to a modern, actively maintained Tomcat release.

FINDING-06 — HSTS Not Set (Allows Downgrade Attacks)

Field	Details
Risk	Low
Confidence	High
Affected URL	All HTTP responses

What is the issue? The application does not return an HTTP Strict-Transport-Security (HSTS) header.

Why does it matter? (Business impact) Even though the site supports HTTPS, a user typing the URL in a browser may first connect via HTTP before being redirected. An attacker on a shared network (coffee shop, airport Wi-Fi) can intercept this initial unencrypted request and strip SSL encryption for the session (*SSL strip attack*), allowing them to eavesdrop on account credentials.

How to fix it: Configure the web server to return `Strict-Transport-Security: max-age=31536000; includeSubDomains` on all HTTPS responses. Also submit the domain to the HSTS Preload List.

FINDING-07 — X-Content-Type-Options Header Missing

Field	Details
Risk	Low
Confidence	Medium
Affected URL	All HTTP responses

What is the issue? The `X-Content-Type-Options` header is absent from server responses.

Why does it matter? (Business impact) Without this header, browsers may "sniff" the MIME type of content and interpret files differently than declared. An attacker who can upload a file (e.g., an image containing script code) could trick a browser into executing it as JavaScript, enabling an XSS-style attack vector.

How to fix it: Add `X-Content-Type-Options: nosniff` to all server HTTP responses.

FINDINGS 08–15 (Low / Informational)

#	Finding	Notes
8	Cookie without `SameSite` attribute	Session cookie `JSESSIONID` lacks `SameSite=Strict`, enabling CSRF via cross-origin requests. Add `SameSite=Strict` to `Set-Cookie`.

#	Finding	Notes
9	Cross-Domain JS inclusion	External JavaScript files loaded from third-party domains. If those domains are compromised, attacker JS runs on the banking site. Review and self-host critical scripts.
10	Secure pages with mixed HTTP content	HTTPS pages loading some resources over HTTP, weakening the security guarantee of HTTPS. Enforce HTTPS for all assets.
11	Suspicious comments in source code	HTML comments contain developer notes (e.g., keywords like `TODO`, internal paths). Strip all comments from production code.
12	Session management identified	Session token (`JSESSIONID`) is visible and transmitted in cookies. Ensure token rotation on login/logout.
13	Cache-control issues	Sensitive pages do not prevent caching, meaning browsers/proxies might cache banking page contents. Apply `Cache-Control: no-store` on authenticated pages.
14	Modern web application	ZAP informational alert confirming the app uses modern frameworks. No action required.

7. Conclusion & Recommendations

Overall Security Rating: MODERATE RISK

The Altoro Mutual application is operational but is running with **significant security debt**. The absence of essential HTTP security headers and the use of a decade-old server component expose the platform to well-documented, easily executed attacks — particularly against its user base.

Priority Remediation Plan

Priority	Action	Effort	Impact
P1	Upgrade Apache Tomcat to latest LTS version	High	Critical
P1	Implement CSRF tokens on all forms	Medium	High
P2	Add `Content-Security-Policy` header	Low	High
P2	Add `X-Frame-Options: SAMEORIGIN` header	Low	High
P3	Add `Strict-Transport-Security` header	Low	Medium
P3	Add `X-Content-Type-Options: nosniff` header	Low	Medium
P3	Disable `PUT` / `DELETE` HTTP methods	Low	Medium
P4	Add `SameSite=Strict` to session cookies	Low	Low–Medium